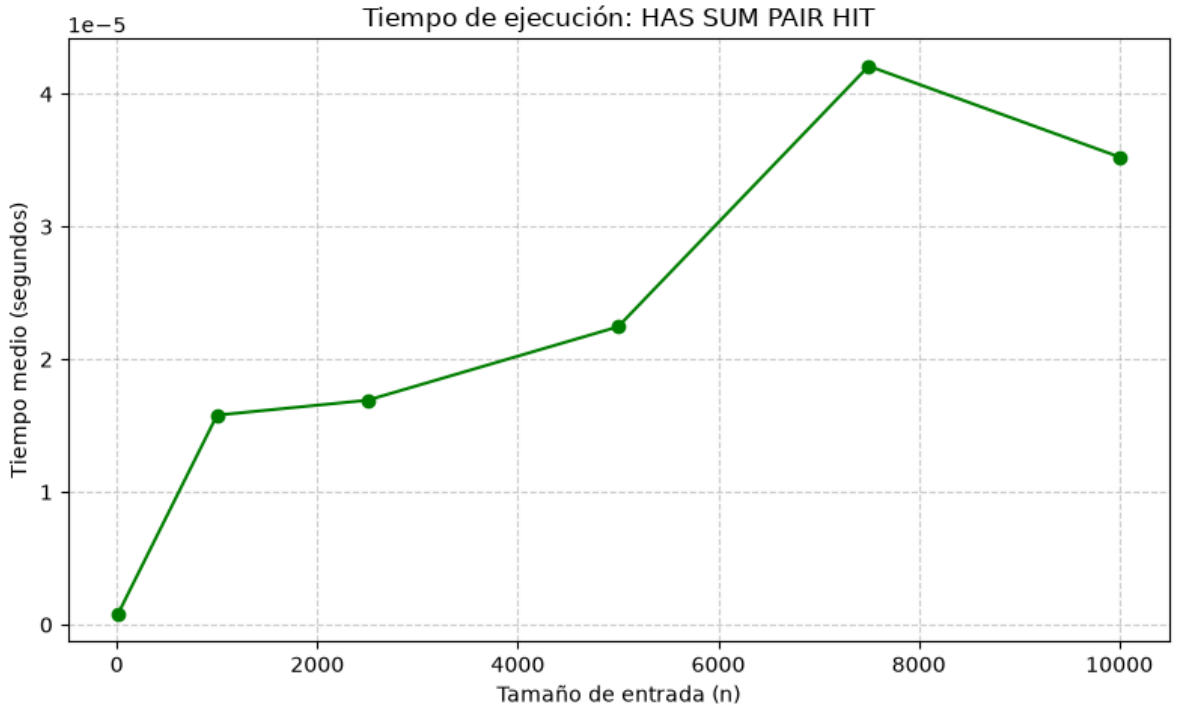
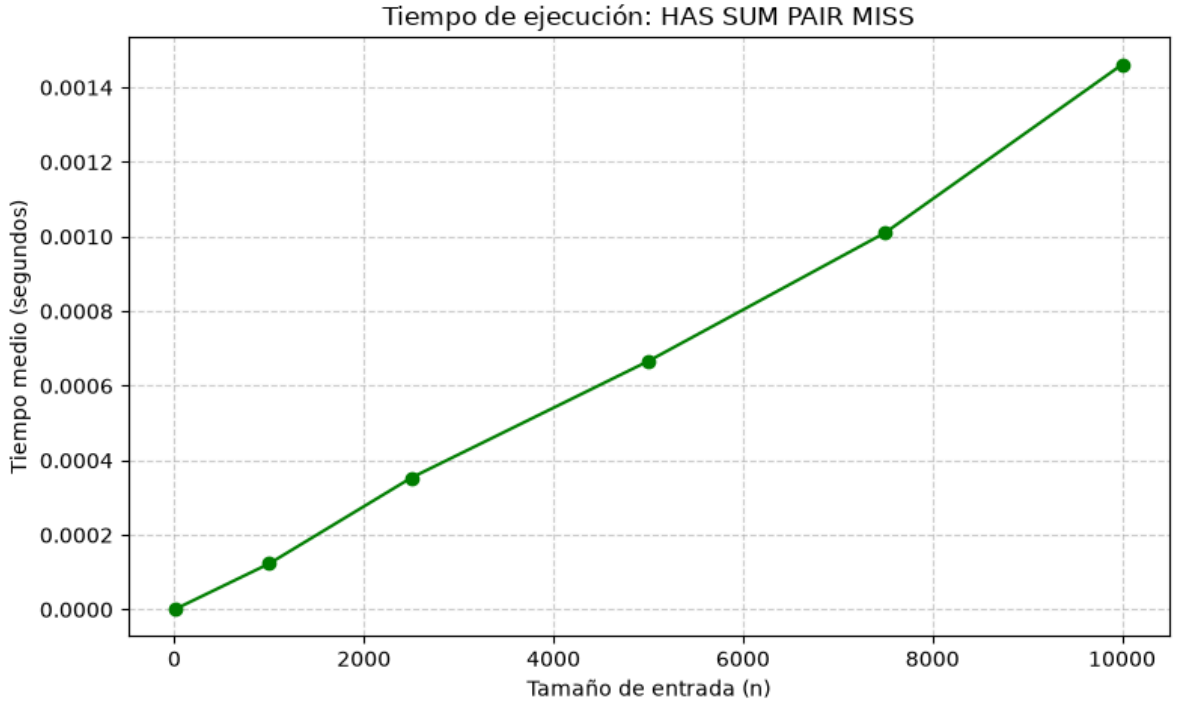
Gráfica Has_sum_pair_hit:



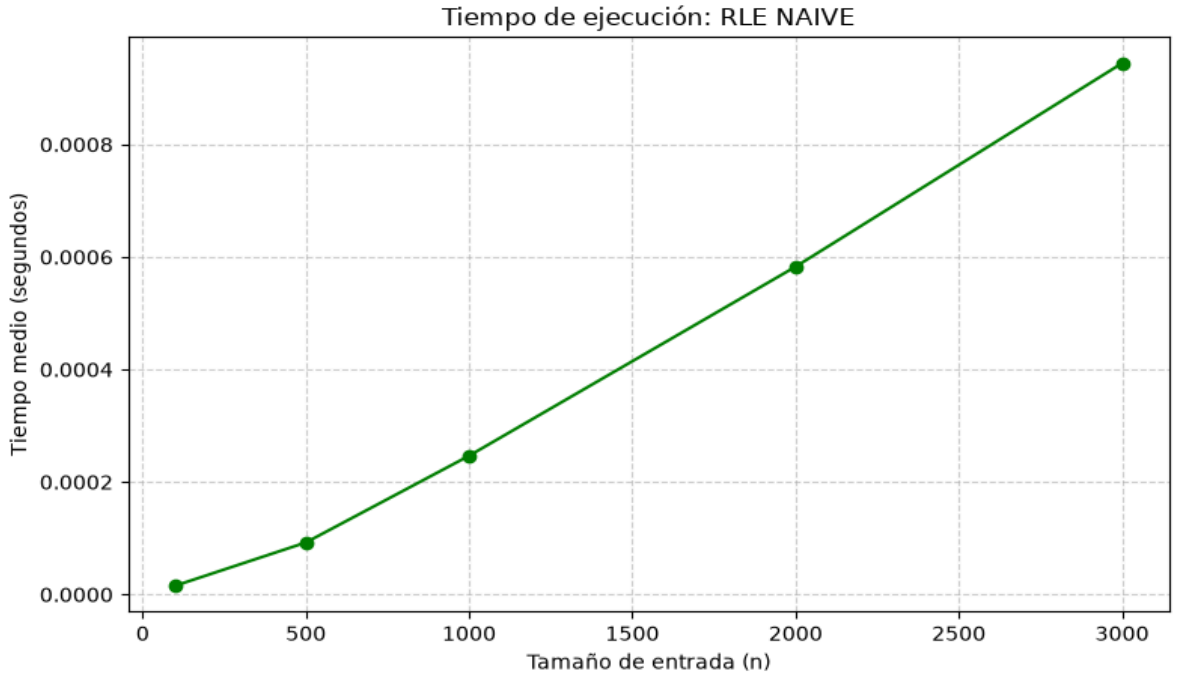
Gráfica Has_sum_pait_miss:



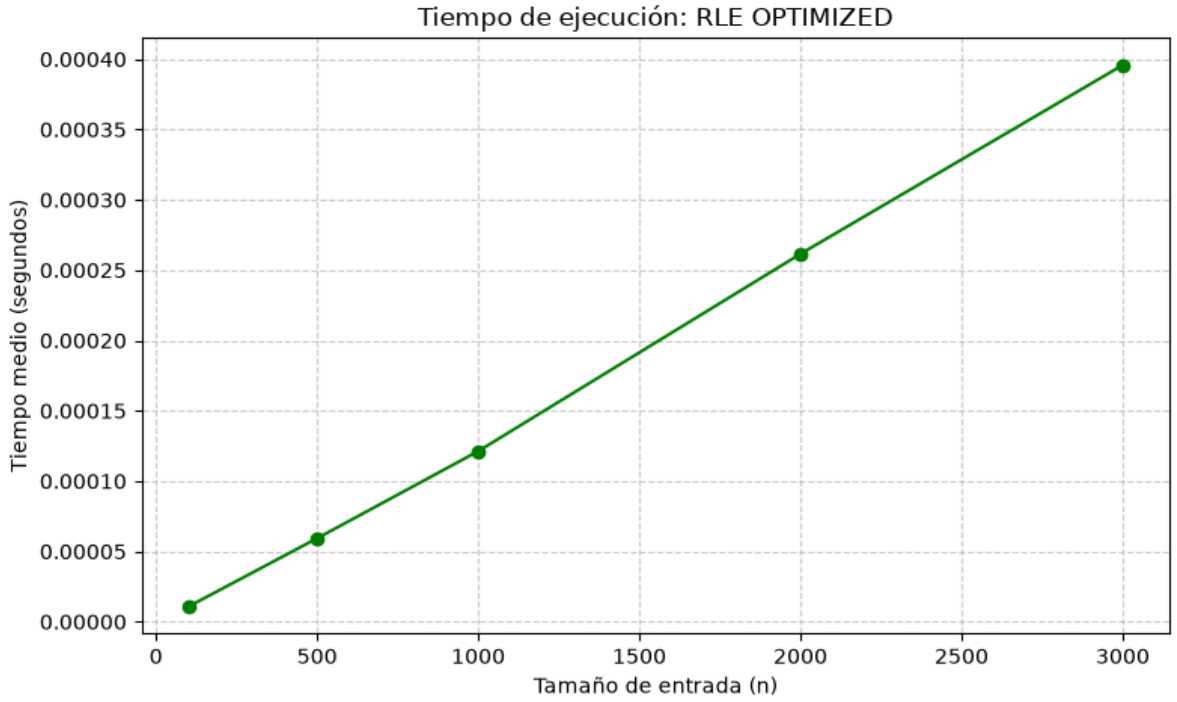
Como podemos observar en las gráficas, la primera tiene un tiempo de ejecución menor y que no aumenta tanto cuando aumentamos el tamaño de entrada, esto se debe a que el tiempo de ejecución varía ya que a veces encuentra la pareja que suma target y termina de buscar.

En cambio en la segunda siempre recorre todas las posibilidades y nunca encuentra la pareja lo que mantiene el tiempo de ejecución mayor y con crecimiento más constante siendo proporcional a n.

Gráfica RLE Naive:



Gráfica RLE Optimized:



En estas gráficas podemos ver la diferencia de tiempo debido a que la concatenación se hace de formas distintas, al usar el operador + (lista = lista + [...]) lo que crea una nueva lista haciendo que eleve la complejidad a O(n²). Al usar .append en la otra función, reduce el tiempo de ejecución considerablemente y la complejidad a O(n)

Cuestiones :

1.¿Afecta al algoritmo si hay aristas repetidas o en orden inverso?

No afecta al resultado final. En un grafo no dirigido la arista(u,v) es equivalente a (v,u) Si una arista ya se procesó o viene invertida, al calcular find(u) y find(v) ambos nodos ya compartirán el mismo representante. La función union detectará que pertenecen al mismo conjunto y la partición permanecerá inalterada.

2.Demostración de corrección del algoritmo:

La pertenencia a una componente conexa es una relación de equivalencia (reflexiva, simétrica y transitiva). Al inicio, cada vértice forma su propia componente conexa disjunta. Por cada arista (u, v), si están en componentes distintas, la arista demuestra que todos los vértices alcanzables desde u son alcanzables desde v, y la operación union fusiona ambas clases de equivalencia. Al procesar todas las ramas del grafo, dos nodos están en el mismo conjunto si y sólo si existe un camino entre ellos.

3.Coste computacional del algoritmo en función de n y l:

Sean n el número de vértices y $m = len(l)$ el número de ramas, la inicialización cuesta $O(n)$, procesar las m aristas con find y union mediante unión por rangos y compresión de caminos toma $O(m \cdot \alpha(n))$, y la conversión final a diccionario (cd2 dict) requiere $O(n \cdot \alpha(n))$, donde α es la función inversa de Ackermann ($\alpha(n) \leq 4$ en cualquier caso práctico).

En consecuencia, el coste computacional total es $O(n + m \cdot \alpha(n))$, lo que equivale en la práctica a un comportamiento lineal $O(n + m)$.